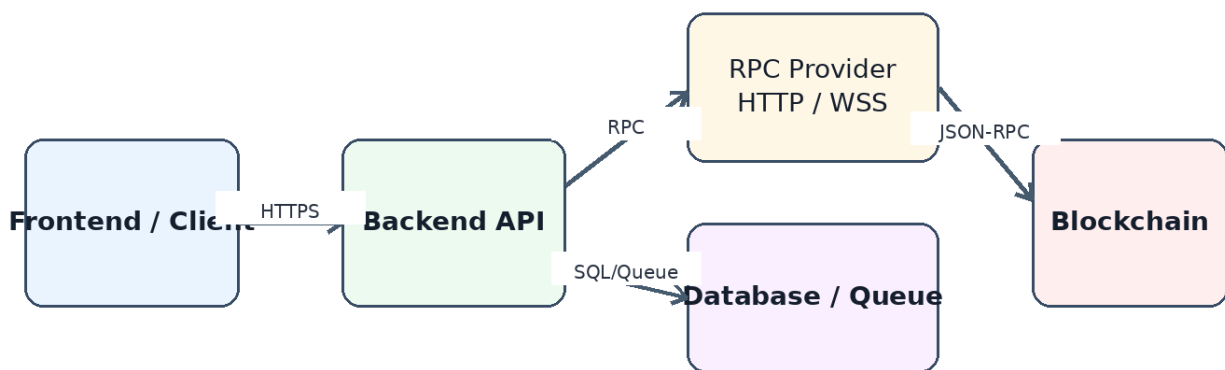


DAY 13

Web3 Backend & Blockchain Automation

Complete 60-Minute Beginner-to-Freelancer Lecture

Web3 Backend Architecture



Goal: Build reliable server-side services that read blockchain data, monitor contract events, persist results, expose APIs, and safely automate transactions.

Course context: Day 12 connected a browser frontend to smart contracts. Day 13 moves the same Web3 interaction into always-on backend services used by real products, monitoring systems, bots, analytics dashboards, payment processors, and client automations.

1. Learning Objectives and 60-Minute Teaching Plan

By the end of Day 13, a beginner should be able to explain the role of a Web3 backend, call an EVM chain through JSON-RPC, read blocks/transactions/receipts, monitor contract events, persist event data, expose an HTTP API, and design an automation worker that is safe against duplicate processing, transient RPC failures, and short chain reorganizations.

Time	Topic	Outcome
0-5 min	Backend mental model	Understand when a dApp needs a server and when it does not.
5-12 min	JSON-RPC	Read the blockchain through standard node methods.
12-18 min	HTTP vs WebSocket providers	Choose transport for reads, polling and subscriptions.
18-25 min	Blocks, transactions and receipts	Build a transaction-inspection backend.
25-32 min	Events and logs	Monitor smart-contract activity.
32-38 min	Backfill + persistence	Store normalized blockchain data without duplicates.
38-44 min	Secure transaction automation	Separate read-only monitoring from signer services.
44-50 min	Reliability engineering	Confirmations, reorgs, retries, idempotency.
50-56 min	API + webhook architecture	Turn blockchain data into business integrations.
56-60 min	Freelancer lab + review	Translate a client request into deliverables and tests.

What you should be able to say after the lecture

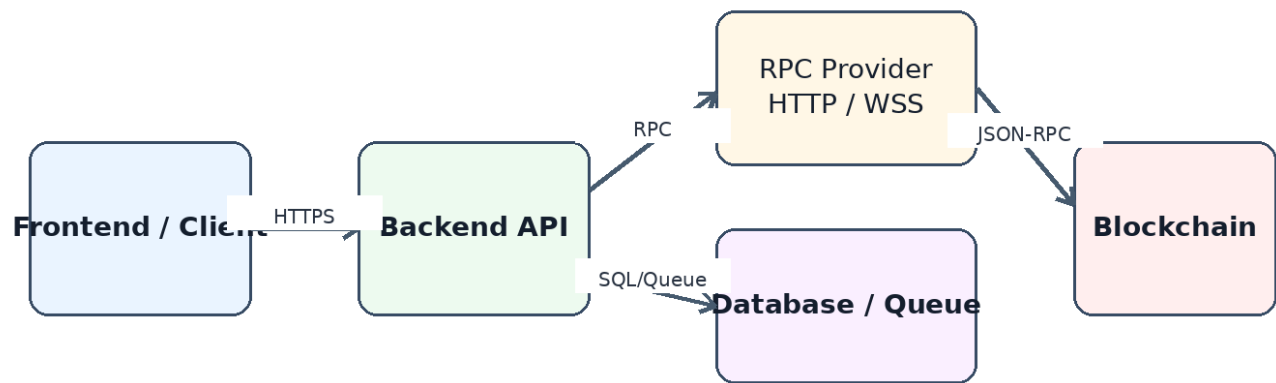
60-second answer: A Web3 backend is a normal server application with blockchain-specific data sources and transaction rules. It talks to nodes through JSON-RPC, decodes ABI-based contract calls and logs, stores normalized results, exposes APIs to other applications, and may sign transactions only when the business workflow requires it. Reliability requires confirmation policies, reorg handling, retries, idempotency, observability, and careful key management.

2. Why Web3 Applications Need Backends

A frontend can connect directly to a browser wallet and blockchain provider, but many production requirements cannot depend on a user keeping a browser tab open. Backends perform continuous, shared, privileged, or data-heavy work.

- 24/7 monitoring of deposits, transfers, mints, swaps or protocol events.
- Historical backfills over thousands or millions of blocks.
- Database indexing for fast dashboards and search.
- Webhooks, email/Slack notifications, accounting exports and CRM integrations.
- Server-side business rules such as “after 12 confirmations, mark invoice paid”.
- Private API credentials and service-to-service authentication.
- Scheduled jobs, queues, retries and reconciliation.
- Transaction sending from controlled service wallets when explicitly required.

Web3 Backend Architecture



3. JSON-RPC: The Backend-to-Node Language

Ethereum execution clients expose a standardized JSON-RPC interface. Your code sends a JSON object containing a method name and parameters; the node returns either a result or an error. Libraries such as ethers.js and web3.py wrap these calls in convenient functions, but professional debugging is easier when you understand the underlying RPC methods.

JSON-RPC: Request -> Node -> Response



3.1 Raw request anatomy

```

POST / HTTP/1.1
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "eth_blockNumber",
  "params": []
}
  
```

3.2 Common methods for backend work

Method	Purpose	Typical use
eth_blockNumber	Latest execution block number	Health checks, sync cursor
eth_getBalance	Native balance	Wallet/account monitoring
eth_getBlockByNumber	Read block	Block ingestion
eth_getTransactionByHash	Read transaction object	Explorer/debugging
eth_getTransactionReceipt	Execution result and logs	Confirmation + event decoding
eth_call	Read contract without state change	Backend queries
eth_getLogs	Historical or filtered logs	Indexing/backfills
eth_estimateGas	Estimate execution gas	Preflight before sending
eth_sendRawTransaction	Broadcast signed bytes	Automation write path

Freelancer habit: When a library call fails, identify which RPC call sits underneath it. This helps distinguish contract bugs from provider limits, malformed parameters, wrong networks, unsupported RPC methods, and rate-limit errors.

4. Providers: HTTP, WebSocket and Polling

A provider is the backend object that knows how to communicate with a blockchain node. Ethers.js v6 provides JsonRpcProvider for HTTP(S) endpoints; event processing on that provider is generally based on polling. WebSocket-based providers can receive subscription-style notifications when the node and infrastructure support them.

Transport	Strengths	Trade-offs	Best beginner use
HTTP/HTTPS	Simple, stateless, widely supported	Events usually require polling	Reads, writes, scheduled polling, backfills
WebSocket	Low-latency pushed notifications	Connections drop; reconnect logic required	Live event/block stream
Hosted provider	Quick setup, managed infrastructure	Quotas, rate limits, vendor behavior	Freelance projects and prototypes
Self-hosted node	Maximum control and data access	Operational cost/maintenance	Advanced production/indexer needs

4.1 Node.js setup - ethers.js v6

```
npm init -y
npm install ethers dotenv express

# .env
RPC_URL=https://your-rpc-endpoint
CONTRACT_ADDRESS=0x...

import "dotenv/config";
import { JsonRpcProvider } from "ethers";

const provider = new JsonRpcProvider(process.env.RPC_URL);
const block = await provider.getBlockNumber();
console.log("latest block:", block);
```

4.2 Python setup - web3.py

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install web3 python-dotenv flask
from web3 import Web3
import os
from dotenv import load_dotenv

load_dotenv()
w3 = Web3(Web3.HTTPProvider(os.environ["RPC_URL"]))
print("connected:", w3.is_connected())
print("latest block:", w3.eth.block_number)
```

5. Read Blockchain Data from a Backend

A backend often starts as a read-only service. This is valuable because read-only monitoring does not require a private key. You can inspect balances, contracts, blocks and transaction receipts while keeping the service incapable of moving funds.

5.1 Node.js transaction inspector

```
import { JsonRpcProvider, formatEther } from "ethers";

const provider = new JsonRpcProvider(process.env.RPC_URL);
const hash = process.argv[2];

const tx = await provider.getTransaction(hash);
if (!tx) throw new Error("transaction not found");

const receipt = await provider.getTransactionReceipt(hash);
console.log({
  hash: tx.hash,
  from: tx.from,
  to: tx.to,
  valueEth: formatEther(tx.value),
  nonce: tx.nonce,
  blockNumber: receipt?.blockNumber,
  status: receipt?.status,
  gasUsed: receipt?.gasUsed?.toString(),
  logs: receipt?.logs?.length ?? 0
});
```

5.2 Python equivalent

```
from web3 import Web3
import os, sys
from dotenv import load_dotenv

load_dotenv()
w3 = Web3(Web3.HTTPProvider(os.environ["RPC_URL"]))
hash_ = sys.argv[1]
tx = w3.eth.get_transaction(hash_)
receipt = w3.eth.get_transaction_receipt(hash_)

print({
  "hash": tx["hash"].hex(),
  "from": tx["from"],
  "to": tx["to"],
  "value_eth": w3.from_wei(tx["value"], "ether"),
  "nonce": tx["nonce"],
  "block_number": receipt["blockNumber"],
  "status": receipt["status"],
  "gas_used": receipt["gasUsed"],
  "logs": len(receipt["logs"]),
})
```

Client use case: A client says “the transaction succeeded, but our backend still marks the payment as pending.” Your first comparison is transaction receipt status + block number + your service’s confirmation cursor + the event your application expects.

6. Transaction Receipts: What Actually Happened?

The transaction object tells you what the sender submitted. The receipt tells you the execution outcome after inclusion in a block. For backend automation, receipts are often more important than the original request because they contain status, gas used and emitted logs.

Field	Meaning	Backend decision
status	1 success, 0 failure on modern EVM chains	Never credit a failed transaction.
blockNumber	Block that included the tx	Start confirmation counting.
gasUsed	Execution gas consumed	Cost analytics/debugging.
logs	Events emitted during execution	Decode business outcome.
contractAddress	Created address for deployment tx	Persist deployed contracts.
transactionHash	Stable tx identifier	Primary external reference.

6.1 Confirmations

A transaction being included in one block is not the same as your business considering it final. Applications choose a confirmation policy based on chain characteristics and value at risk. A backend can calculate confirmations as the difference between the latest block and the receipt block, plus one.

```
const latest = await provider.getBlockNumber();
const receipt = await provider.getTransactionReceipt(hash);
const confirmations = receipt ? latest - receipt.blockNumber + 1 : 0;

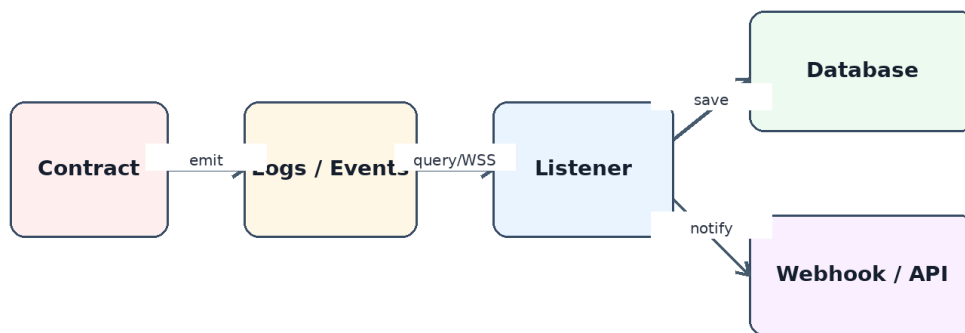
if (receipt?.status === 1 && confirmations >= REQUIRED_CONFIRMATIONS) {
  // safe enough for this application's policy
}
```

Important: There is no universal “correct” confirmation count for every network and every payment value. Treat it as an application risk policy, document it, and make it configurable.

7. Events and Logs: The Backbone of Blockchain Automation

Smart-contract events are written into transaction logs. Backends use these logs to detect business actions efficiently. Instead of repeatedly reading every contract storage variable, an indexer can query relevant log topics or use ABI-aware event APIs.

Event Monitoring Pipeline



Example event

```
event JobPaid(
  uint256 indexed jobId,
  address indexed client,
  address indexed freelancer,
  uint256 amount
);
```

8. Node.js Event Monitoring with ethers.js v6

8.1 Minimal ABI

```
const ABI = [
  "event JobPaid(uint256 indexed jobId, address indexed client, address indexed freelancer, uint256 amount)"
];
```

8.2 Live listener

```
import { Contract, JsonRpcProvider, formatEther } from "ethers";
import "dotenv/config";

const provider = new JsonRpcProvider(process.env.RPC_URL);
const contract = new Contract(process.env.CONTRACT_ADDRESS, ABI, provider);

contract.on("JobPaid", async (jobId, client, freelancer, amount, event) => {
  console.log({
    jobId: jobId.toString(),
    client,
    freelancer,
    amountEth: formatEther(amount),
    txHash: event.log.transactionHash,
    blockNumber: event.log.blockNumber
  });
});
```

With an HTTP `JsonRpcProvider`, ethers can process events through polling. For low-latency streams you may use a `WebSocket` provider, but production code must reconnect and backfill missed blocks after any disconnect.

8.3 Historical backfill

```
const fromBlock = 22000000;
const toBlock = await provider.getBlockNumber();
const filter = contract.filters.JobPaid();
const events = await contract.queryFilter(filter, fromBlock, toBlock);

for (const e of events) {
  console.log(e.blockNumber, e.transactionHash, e.args.jobId.toString());
}
```

Provider limits: Hosted RPC services may restrict the block range or number of logs returned by a single `eth_getLogs` query. A robust indexer chunks the backfill range and persists its last completed block.

9. Python Event Monitoring with web3.py

web3.py exposes contract event APIs and log filters. Infrastructure support varies, so a portable production pattern is often: poll the latest block, request logs for a bounded block range, decode them, persist them, and advance a cursor only after success.

```
import os, time
from web3 import Web3
from dotenv import load_dotenv

load_dotenv()
w3 = Web3(Web3.HTTPProvider(os.environ["RPC_URL"]))
ABI = [{
    "anonymous": False,
    "inputs": [
        {"indexed": True, "name": "jobId", "type": "uint256"},
        {"indexed": True, "name": "client", "type": "address"},
        {"indexed": True, "name": "freelancer", "type": "address"},
        {"indexed": False, "name": "amount", "type": "uint256"}
    ],
    "name": "JobPaid",
    "type": "event"
}]

contract = w3.eth.contract(
    address=Web3.to_checksum_address(os.environ["CONTRACT_ADDRESS"]),
    abi=ABI
)

last = w3.eth.block_number - 10
while True:
    latest = w3.eth.block_number
    if latest > last:
        logs = contract.events.JobPaid().get_logs(
            from_block=last + 1, to_block=latest
        )
        for log in logs:
            print(dict(log["args"]), log["transactionHash"].hex())
            last = latest
    time.sleep(3)
```

Production improvement checklist

- Persist `last` in a database, not only RAM.
- Process in bounded chunks so one giant RPC response does not fail the entire backfill.
- Use a unique key such as `(chain\_id, tx\_hash, log\_index)` for event records.
- Do not advance the cursor until the database transaction commits.
- Handle provider timeouts and retry with exponential backoff.
- Record raw block/log metadata so you can reconcile reorgs.

10. Backfill + Live Tail: The Professional Indexer Pattern

A production event service normally has two jobs: backfill historical data from a known deployment/start block, then continuously ingest new blocks. If the live listener disconnects, the stored block cursor lets the service fill the gap.

```
START_BLOCK = 20_000_000

CHUNK = 2_000

cursor = load_cursor() or START_BLOCK

while True:
    latest = get_safe_head()
    if cursor > latest:
        sleep(POLL_SECONDS)
        continue

    end = min(cursor + CHUNK - 1, latest)
    logs = fetch_logs(cursor, end)

    with db_transaction():
        for log in logs:
            upsert_event(unique_event_key(log), normalize(log))
            save_cursor(end + 1)

    cursor = end + 1
```

10.1 Why “upsert” matters

The same log may be fetched twice because of retries, process restarts or overlap between live and backfill workers. Idempotency means repeating the same input does not create duplicate business effects.

Bad design	Better design
“Whenever I see a log, INSERT a payment.”	Use a unique event identity and UPSERT.
Send webhook before saving database state.	Persist first, then enqueue notification.
Cursor stored only in memory.	Persist cursor atomically with processed events.
One huge range from deployment to latest.	Chunk ranges and retry individual chunks.
Assume logs can never disappear.	Track block hash and support reorg reconciliation.

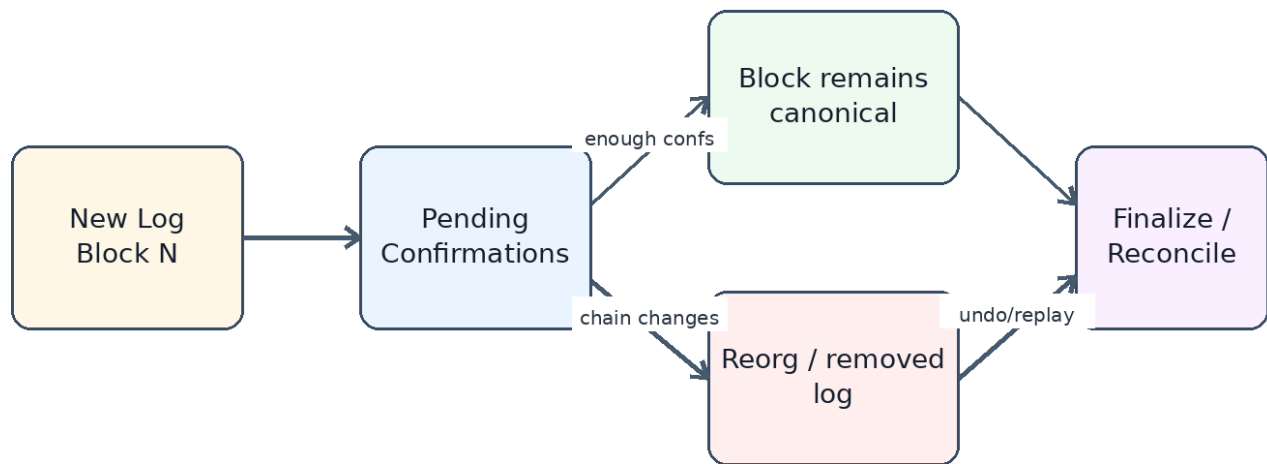
11. Persistence Model

```
CREATE TABLE chain_events (
    chain_id          BIGINT NOT NULL,
    contract_addr     TEXT NOT NULL,
    event_name        TEXT NOT NULL,
    tx_hash           TEXT NOT NULL,
    log_index         INTEGER NOT NULL,
    block_number      BIGINT NOT NULL,
    block_hash        TEXT NOT NULL,
    payload_json      TEXT NOT NULL,
    status            TEXT NOT NULL DEFAULT 'observed',
    created_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(chain_id, tx_hash, log_index)
);
```

12. Reorganizations and Confirmation-Safe Processing

Blockchains can occasionally reorganize: a recently observed block may cease to be part of the canonical chain and be replaced by another branch. A backend that immediately sends irreversible off-chain effects after the first observation can become inconsistent.

Confirmations and Reorg-Safe Processing



12.1 Two-phase business status

```

observed -> confirmed -> finalized-business-action
      \-> orphaned/reconciled
  
```

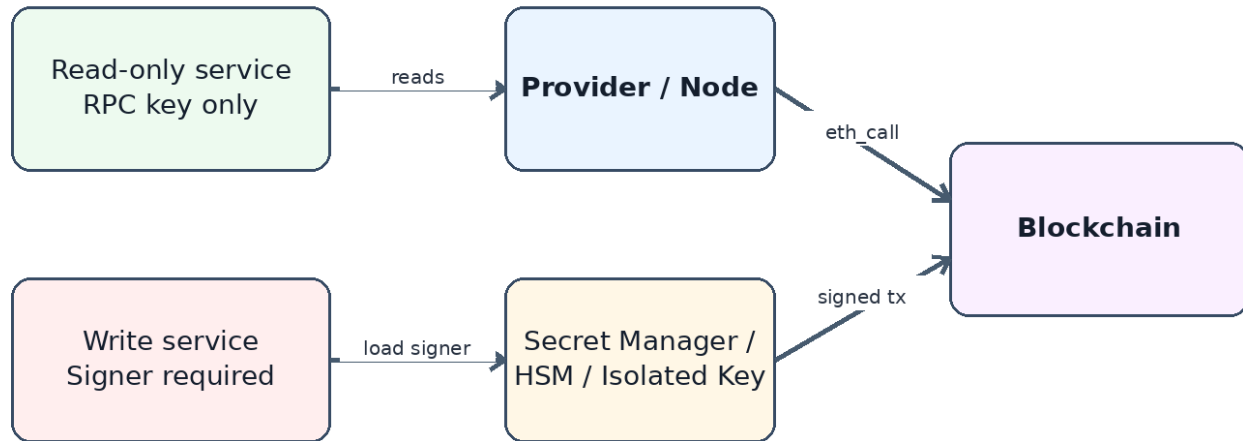
- Store the block hash with each event, not just block number.
- Delay high-value off-chain actions until your confirmation policy is met.
- On each new head, verify recent processed blocks are still canonical.
- If a previously stored event disappears, mark it orphaned and reverse only reversible internal state.
- For external irreversible actions, design compensation workflows rather than pretending perfect atomicity exists across blockchain and Web2 systems.

Freelancer phrase: "I will implement confirmation-aware ingestion and reorg reconciliation" is much stronger than "I will listen for the Transfer event." It demonstrates production thinking.

13. Secure Transaction Automation

Monitoring needs only a provider. Sending a transaction requires a signer. This is a major security boundary. Do not add a private key to a read-only service just because it is convenient.

Read-Only Monitor vs Transaction Signer



13.1 Node.js signer example (test/development pattern)

```

import { JsonRpcProvider, Wallet, Contract, parseEther } from "ethers";
import "dotenv/config";

const provider = new JsonRpcProvider(process.env.RPC_URL);
const signer = new Wallet(process.env.SERVICE_PRIVATE_KEY, provider);
const contract = new Contract(process.env.CONTRACT_ADDRESS, ABI, signer);

const tx = await contract.releasePayment(42, {
  value: parseEther("0")
});
console.log("submitted", tx.hash);
const receipt = await tx.wait();
if (receipt.status !== 1) throw new Error("transaction failed");
  
```

13.2 Production security rules

- Prefer no signer at all for monitoring/indexing services.
- If signing is required, isolate it in a narrowly scoped worker or signing service.
- Use a secrets manager, KMS/HSM or managed signer where appropriate rather than committing keys to files/repositories.
- Use a dedicated service wallet with limited balance and limited on-chain role permissions.
- Use allowlists/business validation before any transaction is signed.
- Never accept an arbitrary destination, calldata or amount from an unauthenticated request.
- Separate testnet and mainnet credentials and configuration.
- Log transaction intent and outcome, but never log private keys or seed phrases.

14. Nonces and Concurrent Transaction Workers

An EOA transaction nonce is sequential. If two backend workers read the same pending nonce and both sign different transactions with it, one may replace the other or fail. This becomes a common client problem when automation scales.

```
# Conceptual nonce allocator
lock(service_wallet)

try:
    nonce = reserve_next_nonce_from_db()
    tx = build_and_sign(nonce=nonce)
    broadcast(tx)
finally:
    unlock(service_wallet)
```

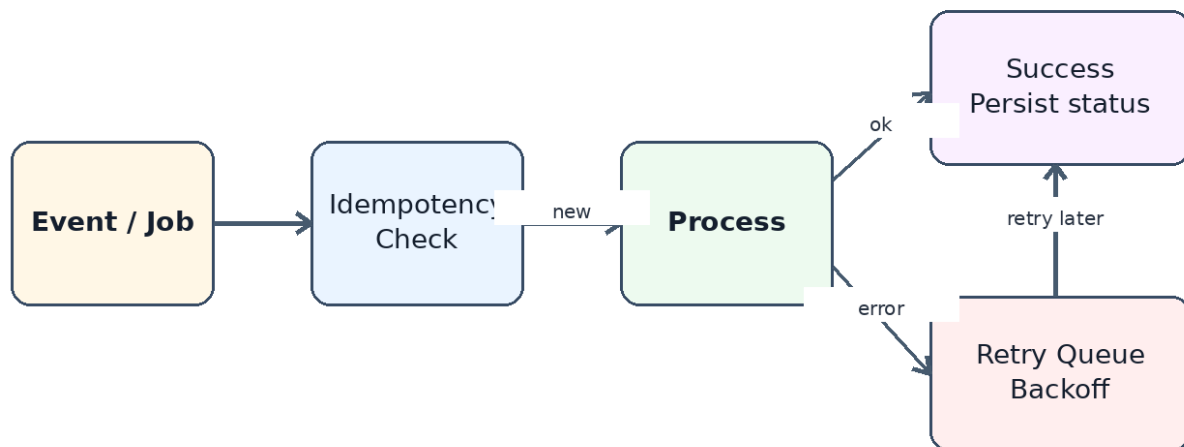
Strategies

- Serialize sends through one queue per signer address.
- Use a database-backed nonce reservation mechanism.
- Track submitted transaction hashes and their nonce.
- Handle replacement/fee-bump policies explicitly.
- Reconcile startup state against both confirmed and pending nonce information.

Beginner warning: Do not “fix” nonce errors by blindly incrementing until the RPC accepts a transaction. Understand what transactions are pending for the address first.

15. Reliability: Retries, Backoff and Idempotency

Reliable Automation Worker



RPC calls can fail for reasons unrelated to your contract: provider rate limits, transient network errors, gateway timeouts or node restarts. Backend code should classify errors instead of retrying everything blindly.

Failure	Retry?	Action
HTTP 429/rate limit	Yes	Backoff with jitter; reduce request rate.
Timeout / connection reset	Usually	Retry bounded number of times.
Invalid params	No	Fix request/configuration.
Contract revert	Usually no	Decode reason; fix business condition.
Insufficient funds	No automatic retry	Alert/fund signer or reduce cost.
Nonce conflict	Maybe	Reconcile pending txs and nonce state.
Database duplicate key	No side effect	Treat as already processed.

16. API Layer: Turn Chain Data into a Product

Once blockchain events are normalized into a database, the rest of your company or client should not need to understand RPC logs. Expose a stable API such as:

```
GET /api/payments/42

{
  "jobId": "42",
  "status": "confirmed",
  "client": "0x...",
  "freelancer": "0x...",
  "amountWei": "1000000000000000000",
  "txHash": "0x...",
  "blockNumber": 23456789,
  "confirmations": 12
}
```

16.1 Express API example

```
import express from "express";
const app = express();

app.get("/api/events/:txHash", async (req, res) => {
  const row = await db.findByTxHash(req.params.txHash);
  if (!row) return res.status(404).json({ error: "not found" });
  res.json(row);
});

app.listen(3000, () => console.log("API on :3000"));
```

16.2 API security

- Validate chain IDs, addresses, hashes, amounts and pagination inputs.
- Rate-limit public endpoints.
- Authenticate administrative or transaction-triggering endpoints.
- Do not expose internal RPC keys in browser responses.
- Never build a “sign arbitrary calldata” endpoint.
- Return stable application statuses instead of raw provider-specific error strings.

17. Webhooks and Business Automation

Many outsourcing projects ask: “When token X arrives, update our database and notify our backend.” The blockchain monitor should create an internal event or queue job, and a separate delivery worker should send webhooks with retries and signatures.

```
Blockchain log
-> normalized DB record
-> enqueue webhook job
-> POST client endpoint
-> expect 2xx
-> retry with backoff on transient failure
```

18. Hands-On Project: Blockchain Event Monitor API

Build a small service that monitors `JobPaid` events, writes unique events to persistent storage, exposes a REST endpoint, and does not require a private key.

18.1 Suggested project structure

```
day13-monitor/
├── src/
│   ├── config.js
│   ├── provider.js
│   ├── abi.js
│   ├── indexer.js
│   ├── repository.js
│   ├── api.js
│   └── main.js
├── data/
├── .env.example
├── .gitignore
├── package.json
└── README.md
```

18.2 `.env.example`

```
RPC_URL=https://...
CHAIN_ID=11155111
CONTRACT_ADDRESS=0x...
START_BLOCK=0
CONFIRMATIONS=3
PORT=3000
```

18.3 Minimal normalized record

```
{
  "chainId": 11155111,
  "event": "JobPaid",
  "jobId": "42",
  "client": "0x...",
  "freelancer": "0x...",
  "amount": "1000000000000000000",
  "txHash": "0x...",
  "logIndex": 7,
  "blockNumber": 6123456,
  "blockHash": "0x...",
  "status": "observed"
}
```

19. Complete Beginner Indexer Algorithm

```
const CONFIRMATIONS = Number(process.env.CONFIRMATIONS ?? 3);
let cursor = await repository.loadCursor();

while (true) {
  try {
    const latest = await provider.getBlockNumber();
    const safeHead = latest - CONFIRMATIONS + 1;

    if (cursor <= safeHead) {
      const end = Math.min(cursor + 499, safeHead);
      const logs = await contract.queryFilter(
        contract.filters.JobPaid(), cursor, end
      );

      await repository.transaction(async () => {
        for (const log of logs) {
          await repository.upsert({
            chainId: Number(await provider.getNetwork()).chainId,
            txHash: log.transactionHash,
            logIndex: log.index,
            blockNumber: log.blockNumber,
            blockHash: log.blockHash,
            jobId: log.args.jobId.toString(),
            client: log.args.client,
            freelancer: log.args.freelancer,
            amount: log.args.amount.toString()
          });
        }
        await repository.saveCursor(end + 1);
      });

      cursor = end + 1;
    } else {
      await sleep(3000);
    }
  } catch (err) {
    console.error(err);
    await sleepWithBackoff();
  }
}
```

Architecture lesson: The exact database library is less important than the guarantees: unique event keys, an atomic cursor, bounded block ranges, retries, and explicit confirmation policy.

20. Testing the Backend

Test	Expected result
Same log ingested twice	One database record; one business effect.
RPC fails for 20 seconds	Worker retries and resumes from persisted cursor.
Process restarts	No historical gap; no duplicate side effects.
Wrong chain ID	Service refuses to start or raises clear alert.
Contract address invalid	Fail-fast configuration validation.
Log at confirmation boundary	Not finalized too early.
Simulated reorg	Old record orphaned/reconciled.
Webhook endpoint returns 500	Retry delivery without re-indexing chain.

21. Debugging Matrix for Client Work

Symptom	Likely causes	First checks
No events detected	Wrong chain/address/ABI/start block	Network chain ID; explorer logs; ABI signature.
Historical query fails	Provider range/result limit	Use smaller chunks; check RPC docs.
Duplicate payments	Non-idempotent processing	Unique key + transaction boundaries.
Missed events after restart	Cursor only in RAM	Persist cursor; backfill gap.
Backend says success but tx reverted	Watching submission not receipt	Check receipt.status and expected event.
Live listener randomly stops	Dropped WebSocket/provider issue	Reconnect and backfill from cursor.
Automation txs replace each other	Nonce concurrency	One signer queue; nonce manager.
Wrong balance/decimals	Raw integer unit mismatch	Read token decimals; store raw units.
Payment credited too early	No confirmation/reorg policy	Add pending -> confirmed lifecycle.

22. Upwork / LinkedIn Freelancer Scenarios

Scenario A - Deposit Monitor

Client request: "Watch our payment wallet for USDC deposits and notify our API."

- Clarify chain, token contract, destination addresses and finality policy.
- Prefer Transfer-event monitoring for the exact token contract; validate `to` and amount.
- Store raw integer amount + token decimals, transaction hash and log index.
- Use confirmation states before calling the client webhook.
- Sign webhook payloads so the client can verify authenticity.

Scenario B - NFT Mint Analytics

Client request: "Track every NFT mint and show dashboard totals."

- Identify mint event convention, often Transfer from zero address for ERC-721.
- Backfill from deployment block.
- Normalize tokenId, recipient, tx hash, block time and collection address.
- Expose aggregated API endpoints instead of making the frontend scan chain history.

Scenario C - Automated Contract Action

Client request: "When condition X occurs, call function Y automatically."

- Separate detection from signing.
- Define authorization and safety checks before signing.
- Queue jobs; use idempotency keys.
- Serialize transactions per signer or implement nonce management.
- Add daily limits/emergency stop when financial risk exists.

23. Freelancer Requirement Checklist

Before giving a fixed-price quote, collect the following information. Missing answers are scope risks.

Area	Questions
Network	Which chain(s), mainnet/testnet, chain IDs?
Contracts	Addresses, ABI, deployment blocks, proxy or implementation?
Events	Exact events/fields? Historical backfill required?
Finality	How many confirmations before business action?
Scale	Expected events/day and historical block range?
RPC	Client provider or developer-provided? Rate limits?
Database	Postgres/MySQL/SQLite/managed service? Existing schema?
Integrations	Webhook/API/Slack/email/CRM/accounting?
Writes	Read-only or server-side transactions? Who controls signer?
Security	Secret manager, authentication, spending/role limits?
Operations	Hosting, logs, alerts, retries, SLA, handover?

Proposal language: “I will deliver a confirmation-aware event indexer with persisted block cursors, idempotent event storage, retry/backoff, REST endpoints and deployment documentation” is a concrete deliverable, not a vague claim of being a Web3 expert.

24. Interview Questions

Q: What is JSON-RPC?

A: A request/response protocol used by applications to invoke node methods such as `eth_call`, `eth_getLogs` and `eth_getTransactionReceipt`.

Q: Provider vs signer?

A: A provider reads/broadcasts network data; a signer can authorize transactions or messages using a key.

Q: Why monitor events?

A: Logs provide efficient, ABI-decodable records of contract activity suitable for indexing and automation.

Q: What is idempotency?

A: Processing the same input multiple times yields the same final business result instead of duplicate effects.

Q: Why persist a block cursor?

A: So ingestion resumes from a known position after crashes/restarts and can backfill gaps.

Q: Why wait for confirmations?

A: Recent blocks may reorganize; applications choose a finality policy based on risk.

Q: Why can WebSocket-only listeners miss events?

A: Connections can drop. A robust service reconnects and queries blocks since its stored cursor.

Q: How do you uniquely identify an EVM event log?

A: Commonly by chain ID + transaction hash + log index, while retaining block hash for reorg checks.

Q: What is a nonce concurrency problem?

A: Multiple senders/workers may try to use the same next EOA nonce, causing replacements or failures.

Q: Why should a monitor not have a private key?

A: Least privilege: read-only work does not need signing capability, reducing blast radius.

25. Day 13 Quiz

1. Name three JSON-RPC methods useful for a backend indexer.
2. What is the difference between a transaction object and a transaction receipt?
3. Why is `(chainId, txHash, logIndex)` useful as a unique event key?
4. Why should a live WebSocket listener also have a backfill strategy?
5. What is a persisted block cursor?
6. What is a chain reorganization?
7. What does idempotency protect against?
8. When does a backend need a signer?
9. Why is blindly retrying a contract revert usually wrong?
10. What problem occurs if two workers use the same EOA nonce?
11. Why should high-value business actions wait for confirmations?
12. Name three secrets/security rules for a transaction-sending backend.

Answer Key

1. Examples: `eth_getLogs`, `eth_getTransactionReceipt`, `eth_getBlockByNumber`, `eth_call`, `eth_blockNumber`.
2. The transaction is the submitted instruction; the receipt records inclusion/execution outcome and logs.
3. It makes repeated ingestion safe by uniquely identifying a log on a chain.
4. WebSocket connections can disconnect, so the service must retrieve missed blocks/events.
5. A durable record of the next or last processed block used to resume ingestion.
6. A recent canonical block is replaced by a different chain branch.
7. Duplicate records and duplicate off-chain/business side effects.
8. Only when it must sign a message or transaction; read-only monitoring does not.
9. A revert is usually deterministic business/contract logic, not a transient network failure.
10. Transactions can conflict, replace one another or fail with nonce-related errors.
11. To reduce the risk of acting on an event that later disappears after a reorg.
12. Examples: secret manager/KMS, least-privilege service wallet, no keys in git/logs, authenticated write endpoints, spending/role limits.

26. Homework and Portfolio Assignment

Homework A - Transaction Inspector

13. Write Node.js or Python code that accepts a transaction hash.
14. Print sender, receiver, value, nonce, receipt status, block number, gas used and number of logs.
15. Add a clear "not found / pending / success / failed" status.
16. Run it on at least three testnet transactions and save example output in README.md.

Homework B - Event Backfill

17. Choose one contract you control on a testnet.
18. Add or identify an event.
19. Query logs from deployment block to latest in chunks.
20. Store a unique event key locally (SQLite/JSON/Postgres is acceptable).
21. Re-run the script and prove it does not duplicate records.

Homework C - API

22. Create `GET /health` showing RPC connectivity, chain ID and latest block.
23. Create an endpoint returning stored events.
24. Add pagination or a result limit.
25. Do not put an RPC key or private key in frontend source code.

Portfolio deliverable

Project: EVM Event Monitor API

README should show:

- architecture diagram
- supported network and contract
- event schema
- confirmation policy
- retry + cursor design
- local setup instructions
- sample API response
- security notes
- screenshots/log examples

27. Day 13 Cheat Sheet

Concept	Remember
JSON-RPC	Standard node request/response interface.
Provider	Reads blockchain and broadcasts data/transactions.
Signer	Authorizes messages/transactions; protect keys.
Receipt	Execution outcome + gas + logs.
Event/log	Efficient contract activity record.
Backfill	Historical ingestion from a block range.
Cursor	Persisted progress marker.
Idempotency	Same event twice, business effect once.
Confirmation	Risk policy before final business action.
Reorg	Recent canonical history can change.
Retry/backoff	For transient failures, not deterministic reverts.
Nonce queue	Serialize or coordinate transactions per signer.
API	Expose normalized app data, not raw provider complexity.
Webhook	Reliable signed notification to external system.

28. Primary Technical References and Day 14 Preview

Use primary documentation when implementing production systems; provider behavior and library APIs evolve.

Ethereum JSON-RPC API: <https://ethereum.org/developers/docs/apis/json-rpc/>

Geth JSON-RPC Server: <https://geth.ethereum.org/docs/interacting-with-geth/rpc>

ethers.js v6 Providers: <https://docs.ethers.org/v6/api/providers/>

ethers.js v6 JSON-RPC Provider: <https://docs.ethers.org/v6/api/providers/jsonrpc/>

web3.py Events and Logs: <https://web3py.readthedocs.io/en/stable/filters.html>

web3.py web3.eth API: <https://web3py.readthedocs.io/en/stable/web3.eth.html>

OpenZeppelin Contracts 5.x: <https://docs.openzeppelin.com/contracts/5.x/>

Day 14 Preview - DeFi Fundamentals

- DEX architecture: factory, pair/pool and router.
- AMM model and constant-product intuition.
- Liquidity providers, reserves and LP positions.
- Swap flow, slippage and price impact.
- Token approvals and router interactions.
- How to read and explain a DEX swap transaction.

Day 13 completion standard: Before moving on, you should be able to build a read-only backend that connects to an EVM RPC, inspects transaction receipts, backfills and monitors contract logs, stores each event once, resumes from a persisted cursor, and explains how confirmations, reorgs, retries and signer security affect production automation.